🧠 What is Backstage?

Backstage is an **open-source developer portal** created by **Spotify**.
It helps organizations **manage all their internal tools, services, documentation, and resources in one place.**

You can think of it as an **"internal app store"** or **dashboard** for developers, where everything they need for development is centralized.
Backstage is a CNCF Incubation project after graduating from Sandbox.

🎯 Why use Backstage?

In large engineering teams, there are **many microservices, APIs, data pipelines, libraries, and tools**.
Developers often spend a lot of time finding things, setting up new projects, or understanding who owns what.

Backstage helps by:

- Centralizing **information** about all services and tools.
- Providing **templates** for creating new projects easily.
- Integrating with CI/CD pipelines, monitoring tools, and documentation systems.
- Making **onboarding new developers** much faster.

🧩 Main Components of Backstage

Here's a breakdown of the key parts of Backstage:

1. Software Catalog

This is the **heart of Backstage**.

It's a directory (or inventory) of all your:

- Services (APIs, backends)
- Websites or frontends
- Libraries
- Data pipelines
- ML models, etc.

Each item is called a **component**, and it has metadata like:

- Name
- Owner/team

- Repository link
- Deployment status
- Dependencies

So, any developer can find information like:

“Who owns this API?” or “Where’s the code for that service?”

---

2. Software Templates (Scaffolder)

This lets you **create new projects quickly and consistently**.

For example:

- You can have a “Node.js microservice” template.
- A developer clicks “Create New” → fills a small form → and Backstage generates the repo with standard setup, CI/CD, and folder structure.

✅ Helps standardize development.
✅ Saves time for onboarding and setup.

---

3. TechDocs (Documentation)

TechDocs is Backstage’s built-in documentation system.

It converts your **Markdown files (like README.md)** from your repo into beautiful docs pages inside Backstage.

This means you can have:

- API documentation
- Setup guides
- Architecture diagrams
  All in one place — and easily searchable.

---

4. Plugins System

Backstage is **plugin-based**, meaning you can add integrations easily.

Some popular plugins:

- GitHub / GitLab
- Jenkins / CircleCI

- Kubernetes
- PagerDuty
- Sentry
- Datadog
- SonarQube
- AWS / GCP / Azure

You can also **build custom plugins** for your company's internal tools.

---

5. Backstage App (Frontend)

The Backstage UI is a **React app**.
It provides the dashboard where developers:

- Browse the catalog
- Run templates
- Read docs
- View service health, etc.

You can customize it to match your company's branding and needs.

🏗️ How Backstage Works (Simple Flow)

1. **Install Backstage** (usually via Node.js and Yarn).
2. **Register components** in the Software Catalog using YAML files (called catalog-info.yaml).
3. **Add templates** for creating new services.
4. **Integrate with tools** like GitHub, Jenkins, Kubernetes, etc.
5. **Deploy Backstage** (often as a web app or container inside your organization's intranet).

⚙️ Example of a Catalog File

Here's an example catalog-info.yaml for a backend service:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: user-service
  description: Handles user authentication and profiles
  tags:
    - backend
    - auth
spec:
  type: service
```

owner: team-user-auth
lifecycle: production
system: user-platform

This file tells Backstage:

- What the service is (user-service)
- Who owns it (team-user-auth)
- What kind of component it is (service)

🚀 Benefits of Using Backstage

Benefit	Description
🧭 Centralized view	All your services, docs, and tools in one place
⚡ Faster onboarding	New developers can find everything easily
🏗️ Consistent standards	Templates ensure uniform project setup
🧰 Integration friendly	Connects with CI/CD, monitoring, and cloud platforms
👨‍💻 Developer productivity	Reduces time spent searching and switching tools

---

🧠 Summary

Term	Meaning
Backstage	Developer portal platform created by Spotify
Software Catalog	Central inventory of all services and tools
Software Templates	Create new projects automatically
TechDocs	Built-in documentation system
Plugins	Extensions for CI/CD, monitoring, cloud, etc.
Goal	Simplify and unify developer experience

Core Features

Backstage includes the following set of core features:

Authentication and Identity - Sign-in and identification of users, and delegating access to third-party resources, using built-in authentication providers.

Kubernetes - A tool that allows developers to check the health of their services whether it is on a local host or in production.

Notifications - Provides a means for plugins and external services to send messages to either individual users or groups.

Permissions - Ability to enforce rules concerning the type of access a user is given to specific data, APIs, or interface actions.

Search - Search for information in the Backstage ecosystem. You can customize the look and feel of each search result and use your own search engine.

Software Catalog - A centralized system that contains metadata for all your software, such as services, websites, libraries, ML models, data pipelines, and so on. It can also contain metadata for the physical or virtual infrastructure needed to operate a piece of software. The software catalog can be viewed and searched through a UI.

Software Templates - A tool to help you create components inside Backstage. A template can load skeletons of code, include some variables, and then publish the template to a location, such as GitHub.

TechDocs - A docs-like-code solution built into Backstage. Documentation is written in Markdown files which lives together with the code.

BACKSTAGE setup

⚙️ 1. Prerequisites (Install Required Tools)

Before installing Backstage, make sure these are installed on your Mac:

✅ **Required:**

- **Node.js (v18 or later)**
- **Yarn (v1.x)**
- **Git**
- **npx** (comes with Node)

---

commands

1. brew install node or (nvm install <version>) → this is used install specific node version . To use this version (nvm use global <v18>)
2. npm install --global yarn
3. brew install git

now create the app

1. npx @backstage/create-app@latest
2. Enter a name for the app [my-backstage-app]: XXXXXXX
3. cd XXXXX
4. yarn start

installing nvm

Step 1: Create NVM directory

mkdir ~/.nvm

Step 2: Add NVM to your shell config

For **zsh** (default on macOS Catalina and later), open ~/.zshrc in VS Code or any editor:

code ~/.zshrc

Add these lines at the end:

```
export NVM_DIR="$HOME/.nvm"
[ -s "/opt/homebrew/opt/nvm/nvm.sh" ] && \. "/opt/homebrew/opt/nvm/nvm.sh"  # This loads nvm
[ -s "/opt/homebrew/opt/nvm/etc/bash_completion.d/nvm" ] && \.
"/opt/homebrew/opt/nvm/etc/bash_completion.d/nvm"  # This loads nvm bash_completion
```

Save and close the file.

Step 3: Reload your terminal

source ~/.zshrc

Step 4: Verify NVM is working

nvm --version

commands-now use nvm

1. nvm install 18
2. nvm use 18
3. node -v # should show v18.x.x

Reinstall Backstage dependencies

1. rm -rf node_modules
2. rm yarn.lock
3. yarn cache clean
4. yarn install

commands for identifying ports running

1. lsof -i -P -n
2. kill -9 <PID>(for killing specific port)

step by step process for github authentication

1) Create a GitHub OAuth App (simplest for local dev)
Go to **GitHub → Settings → Developer settings → OAuth Apps → New OAuth App**.
Fill in:
Application name: Backstage (or anything)
Homepage URL: http://localhost:3000
Authorization callback URL: http://localhost:7007/api/auth/github/handler/frame
Save, then copy the **Client ID** and generate a **Client Secret**.

Step	Command	Purpose
1	yarn install	Install workspace deps
2	yarn --cwd packages/backend add plugin	Add the GitHub auth provider
3	yarn start	Run Backstage

GitHub Actions CI/CD Setup Guide

Overview

This document explains how to create a **CI/CD pipeline using GitHub Actions** to:

1. Build and test a Java application (Maven)
2. Build a Docker image
3. Push the Docker image to Docker Hub

The pipeline runs automatically on code changes and can also be triggered manually.

Prerequisites

Before creating GitHub Actions, ensure:

- A GitHub repository exists
- Java project uses **Maven**
- A valid Dockerfile exists in the repo root
- Docker Hub account is created
- Docker Hub repository is created (example: backstage2k26/cicd-setup)

Repository Structure

Recommended project structure:

```
CiCD-setup/
├── .github/
│   └── workflows/
│       └── ci-cd.yml
├── Dockerfile
├── pom.xml
└── src/
```

GitHub Actions workflows **must** be placed inside:

.github/workflows/

Step 1: Create Workflow File

Create a file:

.github/workflows/ci-cd.yml

Step 2: Define Workflow Triggers

```
name: CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
```

What this means:

- Runs on every push to main
- Runs on pull requests to main
- Can be triggered manually from GitHub UI

Step 3 & 4: Build & Test Job (CI) & Docker Build & Push Job (CD)

- This job compiles, tests, and packages the application.
- Downloads the built artifact
- Builds the Docker image
- Pushes it to Docker Hub

```
name: CI/CD
```

```yaml
on:
  workflow_dispatch:
    inputs:
      publish:
        description: "Publish Docker image?"
        required: true
        default: "false"

jobs:
# ------------------------
# BUILD & TEST
# ------------------------
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - run: mvn clean verify

# ------------------------
# DOCKER PUBLISH (MANUAL ONLY)
# ------------------------
  docker:
    runs-on: ubuntu-latest
    needs: build
    if: inputs.publish == 'true'

    steps:
      - uses: actions/checkout@v4

      # ✅ Debug (safe, does not leak secrets)
      - name: Debug Docker secrets
        run: |
          echo "USERNAME present -> ${{ secrets.DOCKERHUB_USERNAME != '' }}"
          echo "TOKEN present -> ${{ secrets.DOCKERHUB_TOKEN != '' }}"

      # ✅ Fail fast if secrets missing
      - name: Validate Docker secrets
        env:
          DOCKERHUB_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
          DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
        run: |
          if [ -z "$DOCKERHUB_USERNAME" ] || [ -z "$DOCKERHUB_TOKEN" ]; then
            echo "❌ Docker secrets not available"
```

```
      exit 1
    fi

  # ✅ Docker login (unchanged)
  - name: Login to Docker Hub
    uses: docker/login-action@v3
    with:
      username: ${{ secrets.DOCKERHUB_USERNAME }}
      password: ${{ secrets.DOCKERHUB_TOKEN }}

  # ✅ Correct image name
  - name: Build Docker image
    run: |
      IMAGE=${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name }}
      docker build -t $IMAGE:latest .

  - name: Push Docker image
    run: |
      IMAGE=${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name }}
      docker push $IMAGE:latest
```

Step 5: Configure Docker Hub Secrets

GitHub Actions **never stores credentials in code**.

Add Secrets

Go to:

GitHub Repo → Settings → Secrets and variables → Actions

Add the following **Repository secrets**:

Secret Name	Description
DOCKERHUB_USERNAME	Docker Hub username
DOCKERHUB_TOKEN	Docker Hub access token (Read & Write)

Step 6: Dockerfile Example (Java 21)

```
# Use the official OpenJDK 21 image as the base image
FROM eclipse-temurin:21-jdk-jammy

# Set the working directory inside the container
WORKDIR /app

# Copy the Maven project files
COPY pom.xml .
```

```
COPY src ./src

# Build the application using Maven
RUN apt-get update && apt-get install -y maven && mvn clean package -DskipTests

# Expose the port the app runs on (assuming 8080, adjust if needed)
EXPOSE 8080

# Run the application
CMD ["java", "-jar", "target/${{ values.name }}-1.0-SNAPSHOT.jar"]
```

Step 7: Run the Pipeline

Automatic

- Push code to main
- Open a pull request

Manual

1. Go to **GitHub → Actions**
2. Select **CI/CD**
3. Click **Run workflow**

Pipeline Flow Summary

Push / PR / Manual
↓
Build & Test (Maven)
↓
Docker Build & Push (main branch only)
↓
Docker Hub Image Available

Common Issues & Fixes

Issue	Fix
Docker push denied	Check Docker Hub username & token
Secrets not found	Ensure secrets are repo-level
Image not found	Build & push must be in same job
Workflow not running	Check branch & trigger

Conclusion

This GitHub Actions setup provides:

- Reliable CI/CD
- Secure Docker publishing
- Scalable structure for future stages (deploy, scan, release)

HOW TO CREATE TECHDOCS IN BACKSTAGE

Structure

```
content/
├── mkdocs.yml
├── docs/
│   ├── index.md
│   ├── architecture.md
│   ├── cicd.md
│   └── local-setup.md
├── catalog-info.yaml
└── (rest of your code)
```

In mkdocs.yml -→ Example Structure

```yaml
site_name: Java 21 Service Documentation
site_description: Documentation for Java 21 service scaffolded via Backstage
site_author: Platform Team

nav:
  - Home: index.md
  - Architecture: architecture.md
  - CI/CD: cicd.md
  - Local Setup: local-setup.md

plugins:
  - techdocs-core

markdown_extensions:
  - toc:
      permalink: true
  - tables
  - fenced_code
```

To create Techdocs we need necessary steps

first we have to change in app.config.yaml

```
techdocs:
  generator:
    runIn: docker  (change this Docker to local )
```

steps to install mkdocs in local

STEP 1: Install Python (if missing)

- brew install python
- python3 –version or pip3 --version

STEP 2: Install MkDocs + TechDocs plugin

This installs:

- MkDocs
- Backstage TechDocs plugin

- **Add frontend plugin (app)**

  yarn workspace app add @backstage/plugin-techdocs

- **Add backend plugin (backend)**

  yarn workspace backend add @backstage/plugin-techdocs-backend

Command. → **python3 -m pip install --user mkdocs mkdocs-techdocs-core**

STEP 3: Fix PATH (VERY IMPORTANT)

Add to PATH

nano ~/.zshrc

Add this line at the bottom:

export PATH="$HOME/Library/Python/3.9/bin:$PATH"

Save & exit:

CTRL + O → Enter
CTRL + X

Reload:

source ~/.zshrc

STEP 4: Verify MkDocs installation

mkdocs --version

✔ Expected:

mkdocs, version 1.x.x

STEP 5: Build MkDocs

- Go to path (cd examples/java21-template/content/)
- In that run command → **mkdoc build**
- **Yarn start**